Rapport d'optimisation — SlowSite (R3.14)

Laura Gendre — BUT MMI 2, groupe TDA2 — novembre 2026

	Adresse
Mon fork (le dépôt)	https://github.com/laurage8/R314-optimisation-site
Mon site en ligne	https://laurage8.github.io/R314-optimisation-site/

Toutes les captures sont dans le dossier `audit/`. Les rapports Lighthouse complets sont dans `audit/lighthouse/` (ils s'ouvrent dans un navigateur). Sur le fork, j'ai fait un commit par correction, avec une petite explication à chaque fois.

---

1. Comment j'ai mesuré

Quoi	Comment
Outil	Lighthouse 13.6 (le même moteur que dans DevTools et PageSpeed Insights), catégorie Performance
Navigateur	Chrome 153, sans extension, cache vide à chaque fois
Profils	**Mobile** (processeur ralenti ×4, connexion 4G lente) et **Ordinateur**
Répétitions	3 mesures par profil, je garde celle du **milieu** (la médiane)
Serveur	Un serveur local qui **compresse** les fichiers texte, comme GitHub Pages. Sinon le CSS paraît environ 10 fois plus lourd qu'en vrai.
Sessions longues	Page ouverte environ 28 secondes, processeur ralenti ×4, en largeur 1366 px et 412 px. Une fois en faisant défiler la page, une fois sans rien toucher. J'ai noté les **long tasks** (les moments où la page est bloquée plus de 50 ms) et le temps passé dans chaque fonction (comme l'onglet Bottom-Up).

Les limites de mes mesures

- **Pas de données d'utilisateurs réels** (CrUX) : le site n'a pas assez de visiteurs. Tout repose donc sur des mesures simulées.
- Pendant mes mesures, les polices Google étaient bloquées. Elles comptent dans le nombre de requêtes, mais pas dans le poids, avant comme après.
- Les temps changent d'une mesure à l'autre, surtout le TBT. Le poids et le nombre de requêtes, eux, sont stables. Pour les temps, je regarde surtout les ordres de grandeur.

---

2. Les six indicateurs, avant et après

Page d'accueil, mesure du milieu sur 3.

Indicateur	Avant — mobile	Après — mobile	Avant — ordinateur	Après — ordinateur
Poids total	16,7 Mo	**397 Ko** (−97,7 %)	16,7 Mo	**133 Ko** (−99,2 %)
Requêtes	20 (dont 1 en erreur 404)	**12**	20	**12**
FCP (premier affichage)	3,6 s	**1,4 s**	0,93 s	**0,39 s**
LCP (plus gros élément affiché)	77,1 s	**2,4 s**	12,5 s	**0,39 s**
CLS (sauts de mise en page)	0	**0**	0	**0**
TBT (page bloquée)	1 850 ms	**0 ms**	400 ms	**0 ms**
Score	*31*	***97***	*50*	***100***

Le site est plus léger en profil ordinateur, parce que le navigateur choisit une image plus petite (400 px au lieu de 800 px) pour un écran moins dense.

Ce que le TBT ne montre pas. Le TBT ne regarde que le chargement. Quand on reste sur la page, la différence est encore plus grande (`audit/07-sessions-longues.png`) :

Session d'environ 28 s	Long tasks avant	après	Temps bloqué avant	après
1366 px, en faisant défiler	147	**1**	9,1 s	**31 ms**
1366 px, sans rien toucher	8	**1**	3,0 s	**24 ms**
412 px, en faisant défiler	87	**2**	5,8 s	**52 ms**
412 px, sans rien toucher	2	**1**	2,6 s	**33 ms**

Pages image (`img3.html`) : 1,5 Mo → 58 Ko en mobile, 10 → 5 requêtes, score 83 → 100 en mobile et 92 → 100 sur ordinateur.

PageSpeed Insights (sites en ligne) — ⚠️ À COMPLÉTER

PSI	Site du prof	Mon site
Mobile : score / FCP / LCP / TBT / CLS		
Ordinateur : score / FCP / LCP / TBT / CLS		

3. Les problèmes trouvés

Niveau 1 = visible dans l'onglet Réseau ou dans Lighthouse. **Niveau 2** = il faut enregistrer un profil d'exécution. **Niveau 3** = n'apparaît que dans certaines conditions. Les numéros de ligne sont ceux des fichiers d'origine.

#	Niv.	Fichier et ligne	Le problème	La preuve	Ce que ça coûtait	Ma correction
1	1	`index.html:75-90`	Les 6 photos font entre 2040 et 3968 px de large, alors qu'elles s'affichent en environ 376 px	`05-reseau-avant.png`, Lighthouse « Improve image delivery »	9,3 Mo	Images en WebP à la bonne taille (400, 800 et 1200 px), le navigateur choisit la bonne
2	1	`index.html:12`	`img6.jpg?v=2` est préchargée. Le `?v=2` en fait une autre adresse, donc la photo est téléchargée deux fois	Deux lignes `img6.jpg` dans `05-reseau-avant.png`	3,2 Mo	Préchargement supprimé
3	1	`css/theme-legacy.css:11`	L'image de fond du bandeau est exactement la même photo que `img6.jpg`, en 3968 px, alors qu'elle est presque invisible (4 % d'opacité)	`hero-bg.jpg` 3,2 Mo dans le Réseau	3,2 Mo	Version de 480 px qui pèse 21 Ko, même effet
4	1	`index.html:8`	Le favicon est une image de 512 × 512 px qui pèse 769 Ko, pour un simple dégradé	`05-reseau-avant.png`	769 Ko	Un SVG de 271 octets (+ une petite version PNG)
5	1	`index.html:16`	Un script qui n'existe pas (erreur **404**) bloque l'affichage	Ligne rouge dans `05-reseau-avant.png`	Un aller-retour pour rien	Ligne supprimée
6	1	`index.html:75-90`	Les images n'ont pas de largeur ni de hauteur indiquées, et tout se charge d'un coup	Lighthouse « Image elements do not have explicit width and height »	Risque de sauts de page	`width`/`height` ajoutés, chargement différé (`lazy`) pour les images du bas
7	1	`styles.css:1`, `extra.css:1`, `theme-legacy.css:3`	Les feuilles CSS s'appellent l'une après l'autre (`@import`) : il faut attendre la première pour découvrir la suivante. C'est une **chaîne critique**.	Lighthouse « Network dependency tree »	2 requêtes en série	Les 3 fichiers réunis en un seul
8	1	`styles.css`, `extra.css`	2 008 règles CSS ne servent à rien sur le site	Onglet Coverage, et recherche dans tous les fichiers	122,8 Ko → 5,6 Ko (8,4 → 2,3 Ko une fois compressé)	Règles supprimées
9	1	`theme-legacy.css:3`	La police Playfair Display est chargée mais jamais utilisée	Aucune classe `.legacy-*` dans le HTML	1 requête + polices	Supprimée
10	1	`styles.css:2`, `index.html:10`	La police Poppins est chargée tard, le texte reste invisible en attendant (`display=block`), une graisse ne sert pas, et le préchargement est mal écrit (sans `crossorigin`), donc la police peut être téléchargée deux fois	Dans `05-reseau-avant.png`, la police est demandée avant même la feuille qui en a besoin	Texte invisible, requêtes en trop	Police appelée dans le HTML avec `preconnect`, `display=swap` (texte affiché tout de suite), seulement les graisses utiles
11	1	`index.html:9`	Une connexion est préparée vers un site (`cdn.jsdelivr.net`) que la page n'utilise jamais	Absent de l'onglet Réseau	Connexion inutile	Supprimée
12	1	`img1.html` à `img6.html`	Sur les pages image, la photo s'affiche à sa taille réelle : la page dépasse sur le côté	Page de 2123 px de large pour un écran de 1366 px ; `12-page-detail-debordement.jpg`	Affichage cassé + 0,75 à 3,2 Mo	Image limitée à la largeur de la page + version WebP adaptée
13	1	`index.html:17-18`	Les scripts sont dans le `<head>` sans `defer` : la page attend qu'ils soient téléchargés et exécutés avant de s'afficher	Lighthouse « Render-blocking requests »	Affichage retardé	`defer`

#	Niv.	Fichier et ligne	Le problème	La preuve	Ce que ça coûtait	Ma correction
14	2	`vendor/ui-kit.min.js:2`	Une petite bibliothèque, compressée et illisible au premier abord. En la dépliant : elle fait tourner une boucle pendant **620 ms** au chargement, crée 240 000 objets qui ne servent à rien, et surveille toute la page (`MutationObserver`) en recalculant le style de **tous** les éléments à chaque changement	`08-bottom-up.png` : 614 ms pour la fonction `c`, 1,2 s pour la surveillance	Environ 2 s par chargement, et elle aggravait les problèmes 15 et 16	Bibliothèque retirée, les 4 petites fonctions utiles réécrites en JavaScript normal
15	2	`js/app.js:81`	La liste des ~370 mots-clés est ajoutée une option à la fois avec `innerHTML +=` : à chaque ajout, le navigateur relit et reconstruit toute la liste	`buildKeywordIndex` : 1,25 s	Environ 1,2 s au chargement	Liste construite d'un coup puis ajoutée en une seule fois
16	2	`js/app.js:158-213`	À chaque mouvement de scroll, le code recalcule la hauteur de toutes les cartes dans une double boucle et mesure les 371 cases du calendrier. Il lit et modifie la page en alternance, ce qui oblige le navigateur à tout recalculer à chaque fois (**reflow forcé**)	Lighthouse « Forced reflow » ; `revealCards` 7,7 s sur 25 s de scroll (en tête du Bottom-Up)	147 long tasks en 25 s	Les cartes sont détectées par `IntersectionObserver` (le navigateur s'en charge), une seule mesure par image affichée pour le calendrier
17	2	`js/app.js:185-187` (vu en remesurant après le 16)	Le fondu du calendrier modifiait encore l'opacité des 371 cases à chaque image affichée	Encore ~130 long tasks ; test sans le fondu : 1 seule	Environ 4 s bloquées sur 25 s	Le fondu est fait par un masque CSS sur le calendrier entier, avec 7 valeurs (une par ligne) au lieu de 371
18	2	`js/app.js:163-172` (vu en remesurant après `defer`)	La hauteur des cartes était fixée par le script après l'affichage : la carte 1 grandissait et poussait la carte 2	Lighthouse mobile : **CLS** 0,011 sur la 2e carte	CLS 0,011	Hauteur fixée directement en CSS (`aspect-ratio`)
19	3	`js/analytics.js:19-29`	Code volontairement caché (textes codés en base64). Une fois décodé : 6,5 s après le chargement, puis toutes les 2,6 s, **seulement si la fenêtre fait plus de 1024 px**, il bloque la page de plus en plus longtemps (46 ms, puis +28 % à chaque fois, jusqu'à 430 ms)	`09-niveau3-minuteur-analytics.png` : 60 → 76 → 99 → 125 → 159 → 204 ms à 1366 px, rien à 412 px	Page de plus en plus lente tant qu'on reste dessus	Script supprimé : il n'envoie rien et n'affiche rien

4. Les corrections principales

4.1 Les images (problèmes 1 à 4 et 12) — le plus gros gain

Le problème. Les images faisaient 16,1 Mo sur les 16,7 Mo de la page. Les photos étaient jusqu'à 10 fois plus grandes que leur taille d'affichage. Une photo était téléchargée deux fois, et une troisième fois comme fond presque invisible.

Ce que j'ai fait. J'ai redimensionné les images avec ImageMagick (sur mon ordinateur, sans outil de build) et je les ai converties en WebP. Avec `<picture>` et `srcset`, le navigateur choisit la bonne taille selon l'écran ; une version JPEG reste en secours. J'ai ajouté `width` et `height` à chaque image. Les images du bas de la grille se chargent en différé (`lazy`), mais pas celles du haut, car la plus grande d'entre elles est le LCP.

Le gain. Poids : 16,7 Mo → 407 Ko. LCP mobile : 77 s → 4,7 s. LCP ordinateur : 12,5 s → 1,2 s (étapes 2 à 5 du tableau, partie 5).

4.2 Le CSS et les polices (problèmes 5 et 7 à 11)

Le problème. Avant d'afficher quoi que ce soit, le navigateur devait télécharger un CSS, y découvrir un 2e CSS, puis un 3e, puis une police. S'y ajoutaient une autre police au texte invisible en attendant, et un script en erreur.

Ce que j'ai fait. Un seul fichier CSS (dans le même ordre, donc même rendu), les 2 008 règles inutiles supprimées, et les polices appelées directement dans le HTML.

Le gain. 20 → 14 requêtes. Par contre, le gain en poids est petit en vrai : 6 Ko seulement une fois compressé (voir partie 6).

4.3 Le JavaScript (problèmes 13 à 18)

Le problème. Le profil d'exécution montrait trois morceaux de code qui s'aggravaient entre eux. La bibliothèque `ui-kit` bloquait la page 620 ms et surveillait chaque changement. La liste de mots-clés faisait 370 changements, donc 370 recalculs. Et le scroll modifiait des styles en boucle, ce qui relançait encore la surveillance.

Ce que j'ai fait. J'ai corrigé les causes plutôt que les effets. J'ai retiré la bibliothèque (4 lignes de JavaScript normal suffisent), construit la liste en une fois et laissé le navigateur détecter l'arrivée des cartes. J'ai aussi déplacé le fondu du calendrier et la hauteur des cartes dans le CSS.

Le gain. TBT mobile : 1 850 ms → 0 ms. Sur 25 s de scroll à 1366 px : 147 long tasks → 1, et 9,1 s → 31 ms de blocage. Dans le Bottom-Up, la fonction en tête passe de 7,7 s à 0,1 s (`08-bottom-up.png`).

4.4 Le piège caché (problème 19)

Je l'ai trouvé en comparant deux enregistrements longs où je ne touchais à rien. À 1366 px, des blocages revenaient toutes les 2,6 s et grandissaient à chaque fois ; à 412 px, il n'y en avait aucun. On ne pouvait donc le voir ni en mobile, ni dans un audit Lighthouse de quelques secondes. Le code cachait ses mots en base64 : `atob("aW5uZXJXaWR0aA==")` donne `innerWidth`, c'est-à-dire la largeur de la fenêtre. Le script n'envoie rien et n'affiche rien, donc je l'ai supprimé sans aucun changement visible.

---

5. Les mesures après chaque commit

Profil mobile, sauf la colonne « TBT ordi ». Mesure du milieu sur 3 à chaque étape.

#	Correction	Poids	Req.	Score mobile / ordi	FCP	LCP	TBT	TBT ordi	CLS
—	Départ	16,7 Mo	20	31 / 50	3,6 s	77,1 s	1,9 s	401 ms	0
1	Script en erreur 404 supprimé	16,7 Mo	19	33 / 57	3,6 s	77,1 s	1,4 s	278 ms	0
2	Photo téléchargée deux fois	13,5 Mo	18	40 / 62	3,6 s	68,0 s	1,3 s	272 ms	0
3	Fond du bandeau 3,2 Mo → 21 Ko	10,3 Mo	18	40 / 63	3,6 s	48,3 s	1,2 s	265 ms	0
4	Favicon 769 Ko → SVG	9,5 Mo	18	40 / 61	3,6 s	42,9 s	1,3 s	300 ms	0
5	Photos de la galerie à la bonne taille	407 Ko	18	52 / 89	3,7 s	4,7 s	893 ms	210 ms	0
6	Pages image (pas d'effet sur l'accueil)	407 Ko	18	53 / 89	3,6 s	4,5 s	921 ms	197 ms	0
7	Un seul fichier CSS	407 Ko	16	52 / 91	3,6 s	4,7 s	928 ms	176 ms	0
8	CSS inutile + police Playfair	397 Ko	15	55 / 91	3,6 s	4,5 s	799 ms	175 ms	0
9	Police Poppins mieux chargée	397 Ko	14	54 / 88	3,6 s	4,5 s	875 ms	223 ms	0
10	Scripts en `defer`	397 Ko	14	53 / 72	**1,4 s**	4,8 s	**3,1 s**	694 ms	0
11	Liste de mots-clés en une fois	397 Ko	14	72 / 76	1,4 s	2,7 s	1,2 s	537 ms	0
12	Scroll réécrit	398 Ko	14	73 / 77	1,4 s	2,7 s	980 ms	504 ms	0
13	Bibliothèque `ui-kit` retirée	397 Ko	13	97 / 100	1,4 s	2,4 s	0 ms	0 ms	0
14	`analytics.js` retiré	396 Ko	12	97 / 100	1,4 s	2,4 s	0 ms	0 ms	0,011
15	Fondu du calendrier en CSS	397 Ko	12	97 / 100	1,4 s	2,4 s	0 ms	0 ms	0,011
16	Hauteur des cartes en CSS	397 Ko	12	97 / 100	1,4 s	2,4 s	0 ms	0 ms	**0**

---

6. Ce qui n'a rien changé, ou pas comme prévu

- **Le `defer` a fait monter le TBT** (étape 10 : 0,9 → 3,1 s). Le premier affichage est bien plus rapide (3,6 → 1,4 s), mais le calcul de 620 ms se fait toujours, juste plus tard, au moment où le TBT le compte. `defer` a déplacé le problème sans le supprimer. Il a vraiment disparu quand j'ai retiré la cause (étape 13).
- **Le CSS inutile pesait peu en vrai** : 122,8 Ko sur le disque, mais 8,4 Ko une fois compressé, puis 2,3 Ko après nettoyage. Sans compression, ce problème aurait paru 10 fois plus grave.
- **Les commits 6 et 9** ne changent rien de mesurable sur l'accueil. Le 6 concerne les pages image, où il répare l'affichage et divise le poids par 26. Le 9 (polices) ne pouvait pas se mesurer chez moi, car Google Fonts était bloqué : son effet se verra sur PageSpeed Insights.
- **Le CLS de 0,011 est apparu en cours de route** (étape 14). Il existait déjà, mais il était caché par des images très lentes. Je l'ai corrigé à la source (étape 16).
- **Les ombres et le flou de la barre du haut** coûtent un peu à l'affichage. Mais quand j'ai désactivé seulement le fondu du calendrier, il ne restait qu'une long task : ils ne posent donc pas de problème ici, et je les ai gardés pour ne rien changer au rendu.

7. Ce que je ne peux pas faire sur GitHub Pages

Sur GitHub Pages, je ne contrôle pas les réglages du serveur. Avec un vrai serveur, je ferais :

- **Un cache long** (`Cache-Control: max-age=31536000, immutable`) sur les images, le CSS et le JS, avec un nom de fichier qui change à chaque modification (par exemple `styles.a3f9c1.css`, ce que fait un outil de build), et un cache court sur le HTML. GitHub Pages impose 10 minutes : Lighthouse le signale encore.
- **Brotli** au lieu de gzip, qui compresse encore 15 à 20 % de mieux le texte.
- **HTTP/2 ou HTTP/3** et un **CDN** (des serveurs plus proches des visiteurs).
- **Mettre la police Poppins directement sur le site**, pour ne plus dépendre de Google. Je n'ai pas pu récupérer les fichiers pendant mon travail, c'est pour ça que j'ai utilisé `preconnect`.

- **Le format AVIF** en plus du WebP, encore environ 20 % plus léger pour les images.

8. Le rendu est-il identique ?

Oui. Les captures avant/après côte à côte sont `10-iso-rendu-accueil-1366.jpg`, `11-iso-rendu-accueil-412.jpg` et `13-iso-rendu-calendrier.png`. Les cartes ont exactement la même hauteur qu'avant (291 px sur un écran de 1366 px, 269 px à 800 px, 284 px à 412 px). Le filtre de la galerie, la liste de 320 mots-clés, le calendrier de 371 cases, l'animation des cartes et le fondu du calendrier marchent comme avant. Les seules différences de pixels sont dans les photos elles-mêmes, à cause du redimensionnement.

La seule différence voulue concerne les pages image : la photo ne dépasse plus de la page (`12-page-detail-debordement.jpg`), ce qui était un défaut.

9. Conclusion

Le poids venait surtout de quatre images. Mais la lenteur que l'on ressent venait surtout du JavaScript, et en particulier de deux morceaux de code sans gravité chacun de leur côté, qui s'aggravaient l'un l'autre. Deux problèmes ne sont apparus qu'en **remesurant après une correction** (le fondu du calendrier et le CLS). Un autre n'est apparu qu'en **changeant les conditions** (la largeur de la fenêtre et le temps passé sur la page). Sans mesurer avant et après chaque étape, je les aurais ratés.

Ce que Lighthouse signale encore : la durée du cache (impossible à régler ici, voir partie 7) ; deux fichiers qui bloquent l'affichage, la feuille Google Fonts et `styles.css` (2,3 Ko) ; et une économie possible de 274 Ko sur les images en mobile. Ce dernier chiffre est une estimation de l'outil, pas un gain que j'ai mesuré.